自由制作課題 (個人)

Java Programming Practice: Markov Chain

内山 航平 25A4111026

2026 年 1 月 22 日

目次

0.1 概要 1
0.2 実行例 2
0.3 プログラム解説 3
0.3.1 ファイル構成 3
0.3.2 Main.java 3
0.3.3 Preparation.java 5
0.3.4 Leaning.java 5
0.3.5 Generate.java 6
0.3.6 Probability.java 6
0.4 Eclipse 上でのソースコード一覧 7
0.5 Eclipse 上での実行例 11
0.6 全体的な工夫点 11
0.7 課題 12

0.1 概要

この java プロジェクトは任意の英語文字列を学習し、マルコフ連鎖を用いて文を任意の単語数で生成するものです。

0.2 実行例

```
[STEP 1] Dataset Selection
----------------------------------------
Chose a dataset type:
  0: [ALL]    - Combine all available data
  1: [NORMAL]     - Starndard data set
  2: [SF]     - SF dataset
  3: [SHIKA]      - shikanoko special dataset
  4: [NORMAL2]    - Nature and Environment
  [CUSTOM]    - custom dataset path (numbers prohibited)

Select number (or input path)> 3

----------------------------------------
[INFO] Loading dataset...
  - Path: /home/coyuki/Develop/Java/markov_chain_practice/dataset/shika.txt

[OK] Loaded 14 words successfully

[PROCESS]
  - Analyzing loaded text... [DONE]
  - Calculating probablities... [DONE]

----------------------------------------
Markov Chain Model is ready!
----------------------------------------

[CONFIG]
  - Dataset: shika.txt

How long to generate> 14
Seed word ("***" or not found = random)>  し
How many generations?>

[RESULT]
----------------------------------------

し か の こ し か の こ し か の こ の こ 。

----------------------------------------

[STATUS]
  - Words generated: 14
  - Count: 1
  - Status: Success
  - Seed word: "し"
```

0.3 プログラム解説

まず全体は以下のような流れで処理が行われます。以下、ソースコードから一部抜粋しつつ解説を行う。また抜粋の影響でインデントが崩れるが可読性を優先する。

*下記の図は gemini にソースコードを読ませて生成した図式である。

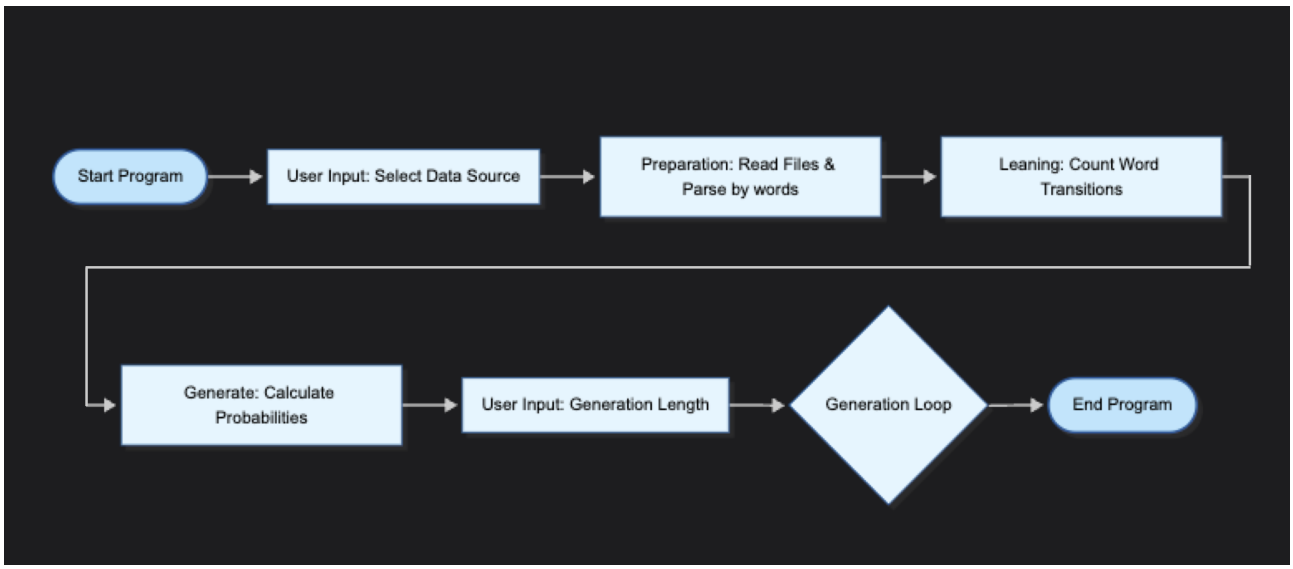


図 1　process of this project

0.3.1 ファイル構成

```
.
├── dataset
│   ├── normal.txt
│   ├── normal2.txt
│   ├── sf.txt
│   └── shika.txt
└── src
    └── markov_chain_practice
        ├── Generate.java
        ├── InvalidInput.java
        ├── Learning.java
        ├── Main.java
        ├── Preparation.java
        ├── PreparationEn.java
        └── Probability.java
```

0.3.2 Main.java

16 行目から 51 行目において学習用データの決定を行なっている。デフォルトで用意したセット (0~4) を使うかどうかをユーザーに選択させるプロンプトを行い、Scanner で結果を取得する。数字以外を入力した場合は、その文字列をファイルパスとして直接読み込む。

`Main.java`

```
var input = sc.nextLine();

// 入力値が int に変換出来ればデフォルトのデータセットを選択
// 失敗すれば入力をそのままパスとして取り込む
var lst = parse_option(input)
        .map(n -> {
            try {
                return switch (n) {
                    case 1 -> List.of(Path.of(default_pare().toString(), "normal.txt"));
                    case 2 -> List.of(Path.of(default_pare().toString(), "sf.txt"));
                    case 3 -> List.of(Path.of(default_pare().toString(), "shika.txt"));
                    case 4 -> List.of(Path.of(default_pare().toString(), "normal2.txt"));
                    default -> get_all_dataset_path();
                };
            } catch (Exception e) {
                throw new RuntimeException(e);
            }
        }).orElse(List.of(Path.of(input)));
```

53 行目から 75 行目において Preparation, Leaning, Generate Class を用いて、入力データの解析を行なっている。解析についての詳細は各 Class の解説を参照。

現在の処理状況を [INFO], [OK], [PROCESS] などのタグと共に詳細に出力するようにし、読み込み済みのパス一覧や単語数を確認できるようにしている。

Main.java

```
// Preparation にてデータの取得と単語へのパースを行う
var parsed_data = new Preparation(lst.toArray(Path[]::new));
var loaded_num = parsed_data.LoadedWordsNum();
var loaded_path_lst = parsed_data.LoadedPathList();

// ... 省略 ...

// Leaning にて各単語の出現回数をカウント
var leaned_data = new Leaning(parsed_data.get_input());

// Generate にて出現回数から次単語の出現確率を各ベクトルに変換
var caled = new Generate(leaned_data);
```

82 行目から 121 行目にかけては、実際の出力を行っている。生成する単語数、開始単語 (Seed)、および生成回数をユーザーに入力してもらい、それに基づいて指定回数分の文生成を行っている。Seed が見つからない場合や” *** “が入力された場合はランダムな単語から開始する。

Main.java

```
// Config 入力
System.out.printf("Seed word ("***" or not found = random)> ");
var in_seed = sc.nextLine();

// ... Seed の決定ロジック ...

System.out.printf("How many generations?>");
var in_num = sc.nextLine();

// 生成ループ
for (int j = 1; j < nn + 1; j++) {
    // i は生成する単語数
    var gened_words_count = caled.loopShow(seed, i, 0);
    // ... ステータス表示 ...
}
```

124 行目以降はエラーハンドリングとして例外を catch し、[ERROR] タグと共に内容を表示して `Exit(1)` で終了する。

0.3.3 Preparation.java

パスからファイル内容の取得と、正規表現を用いた不要な空白や空行の除去、および単語分割を行う。

`Preparation.java`

```
public Preparation(Path[] p) throws Exception {
    try {
        this.loaded_path_lst = new ArrayList<Path>();
        this.input = this.Parse(this.get_some_input(p));
    } catch (Exception e) {
        throw e;
    }
}
```

1 つのファイルの内容を取得し、テキストのクリーニングを行う関数。

`Preparation.java`

```
String get_input(Path p) throws Exception {
    if (!Files.isRegularFile(p)) {
        throw new InvalidInput(String.format("invalid input: %s", p));
    }
    try {
        var s = Files.readString(p)
                .replaceAll(" {2,}", " ")
                .replaceAll("(?m)^[ \t]*\r?\n", "");
        this.loaded_path_lst.add(p);
        return s;
    } catch (Exception e) {
        throw e;
    }
}
```

`\s+`（1 つ以上の空白文字）で分割することで、より堅牢な単語パースを実現している。

`Preparation.java`

```
String[] Parse(String input) {
    return input.split("\\s+");
}
```

0.3.4 Leaning.java

`HashMap<現在の単語，HashMap<次の単語，出現回数>>`となるようにカウントを行う。

`Leaning.java`

```
void collect_analyzer(String[] words) {
    for (int i = 0; i < words.length - 1; i++) {
        this.WordsMap.computeIfAbsent(words[i], k -> new HashMap<>()).merge(words[i + 1].trim(), 1,
            ↪  Integer::sum);
    }
}
```

0.3.5 Generate.java

出現回数から確率（Probability）を計算し、乱数を用いて次単語を選択する。

■0.3.5.1 次単語の生成ロジック Next(String k) 関数では、現在の単語 k に紐づく次単語候補の確率分布から、0 以上、分母未満までの乱数を生成し、累積確率に基づいて次の単語を決定する。

Generate.java

```
public String Next(String k) {
    var start = this.ProbabilityMap.get(k);
    if (start == null) return ".";

    var w = get_sum(start);
    var pivot = RANDOM.nextLong(w);

    var cSum = 0L;
    for (Entry<String, Probability> p : start.entrySet()) {
        cSum += p.getValue().get_num();
        if (pivot < cSum) return p.getKey();
    }
    return ".";
}
```

また、再帰的に loopShow を呼び出すことで指定された長さの文を生成する。

Generate.java

```
public Integer loopShow(String k, int c, int cc) {
    if (c <= 0 || k.equals(".")) {
        System.out.printf(".\n");
        return cc;
    }
    System.out.printf("%s ", k);
    return loopShow(this.Next(k), c - 1, cc + 1);
}
```

0.3.6 Probability.java

分母と分子を保持し、toString() で分数形式のデバッグ出力を提供する。

Probability.java

```
public class Probability {
    long num; // 分子
    long den; // 分母

    public Probability(long n, long d) {
        this.num = n;
        this.den = d;
    }
    // ... getter ...
}
```

0.4　Eclipse 上でのソースコード一覧

*注: コードの更新に伴い、以下のスクリーンショットの内容は現在の最新コードと一部異なる場合があります。

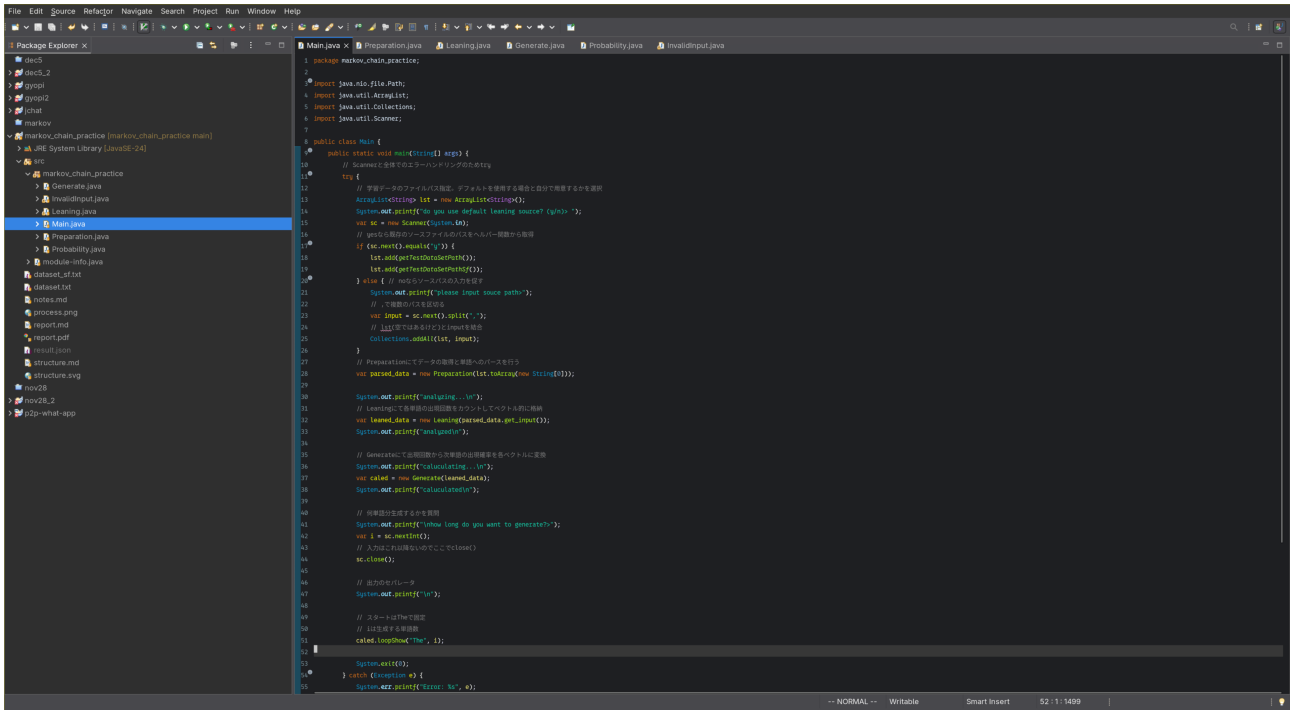


図 2　main.java-1

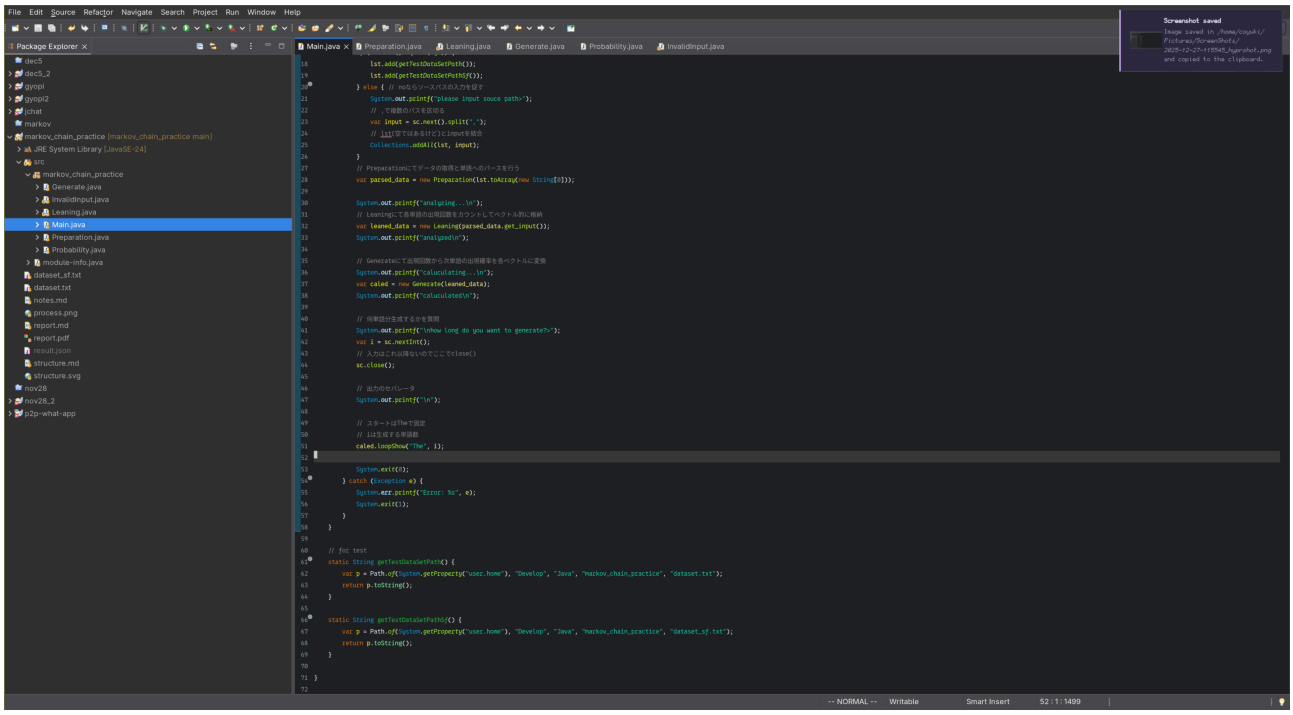


図 3　main.java-2

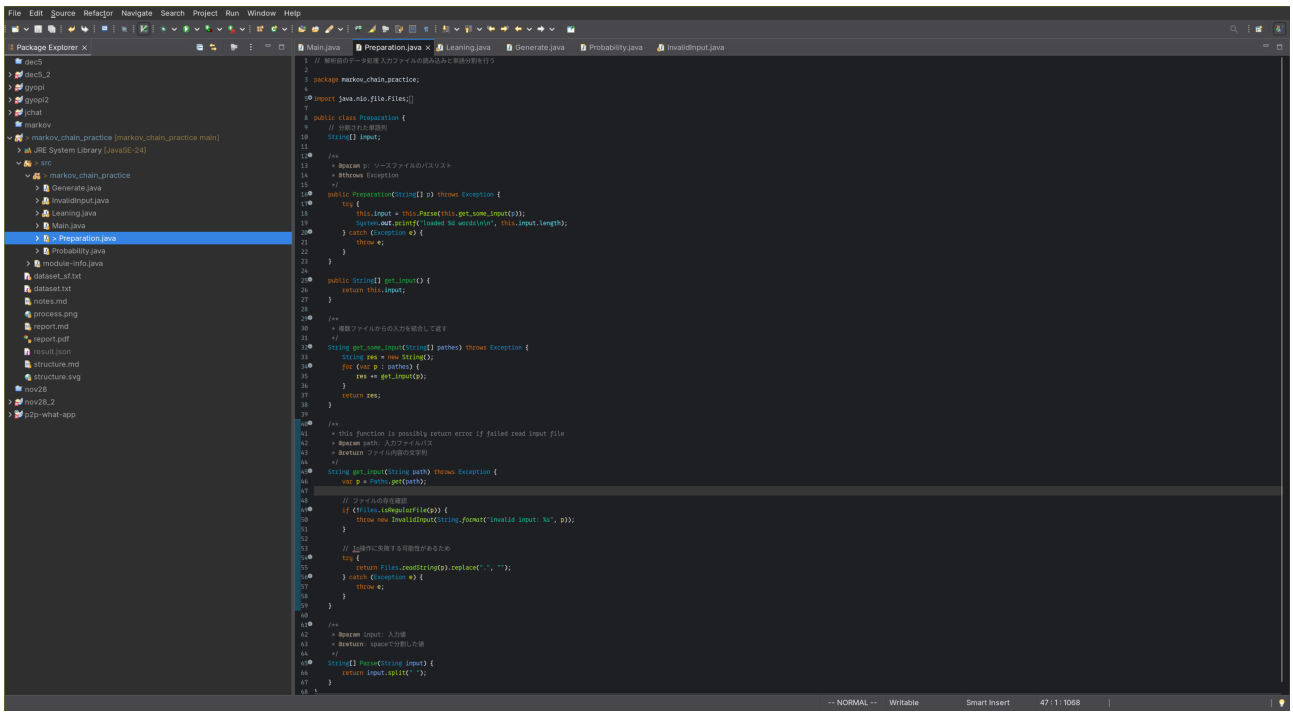


図 4　Preparation.java

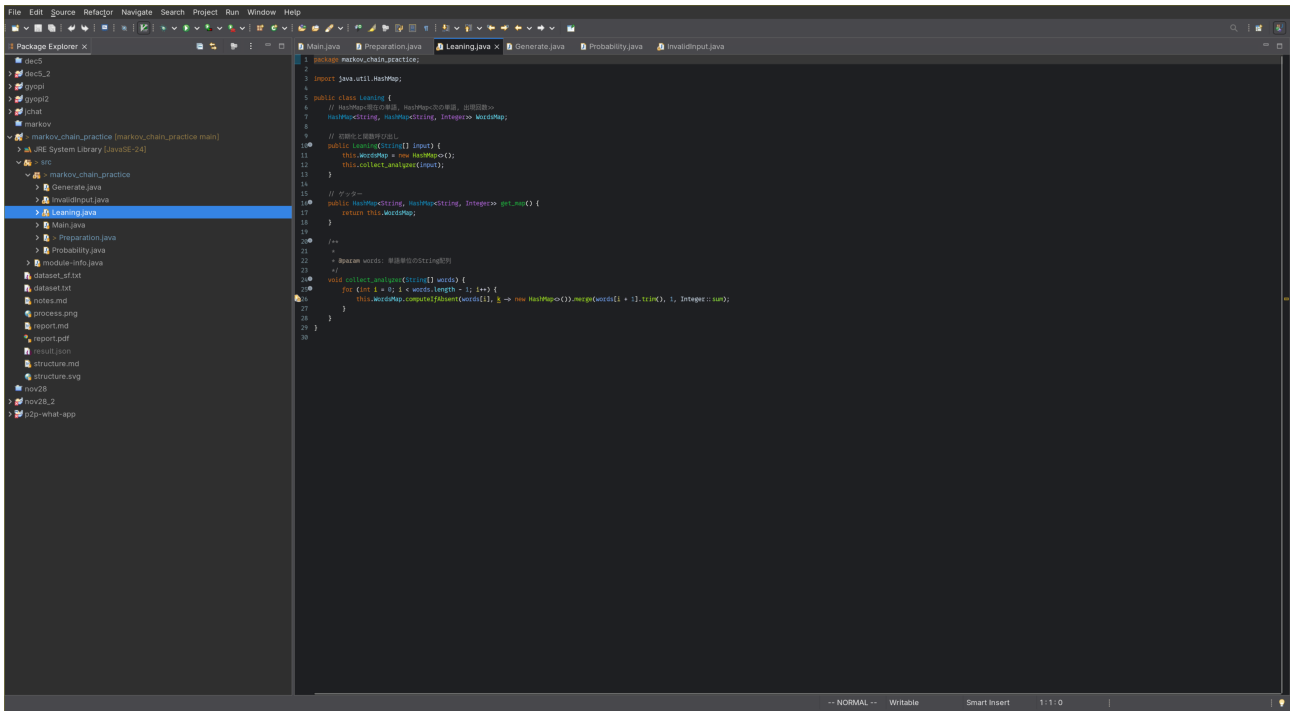
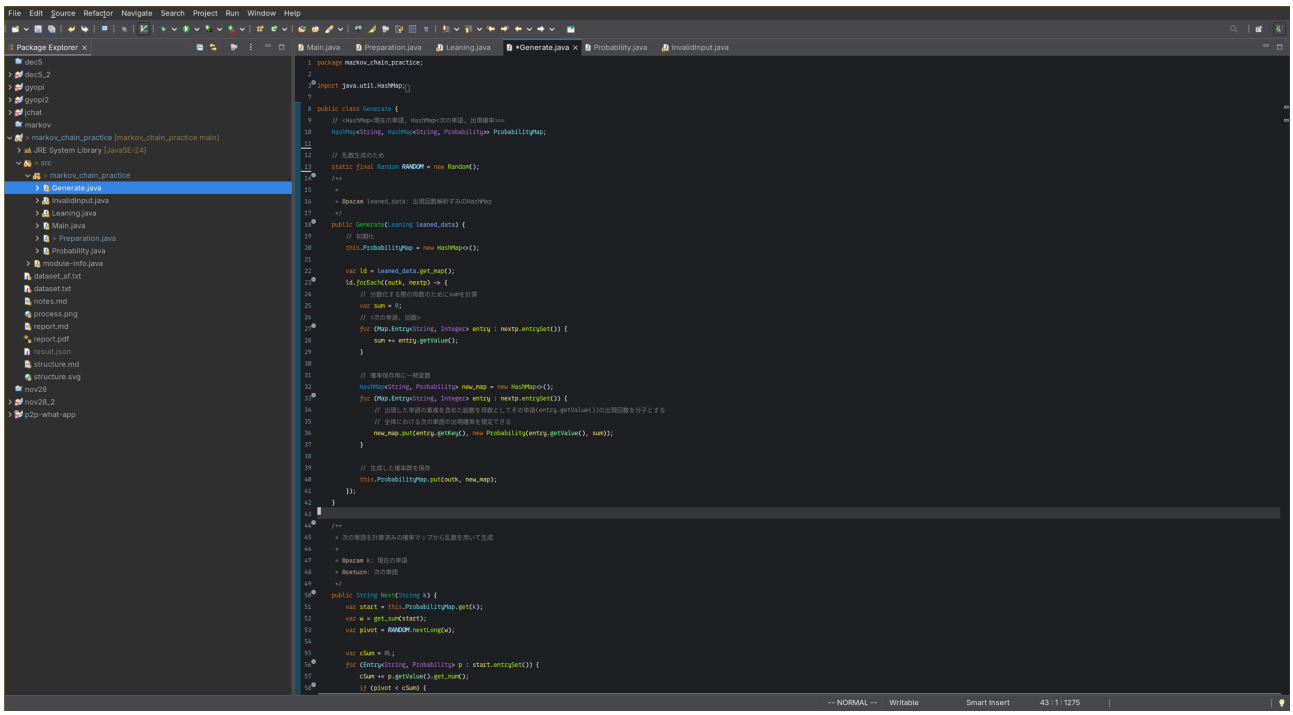


図 5　Leaning.java



```java
package markov_chain_practice;

import java.util.HashMap;

public class Generate {
    // chainMap=現在の単語, HashMap<次の単語, 出現確率>>
    HashMap<String, HashMap<String, Probability>> ProbabilityMap;

    // 乱数生成のため
    static final Random RANDOM = new Random();
    /**
     *
     * @param leaned_data: 出現回数解析ずみのHashMap
     */
    public Generate(Leaning leaned_data) {
        // 初期化
        this.ProbabilityMap = new HashMap<>();

        var ld = leaned_data.get_map();
        ld.forEach((outk, nextp) -> {
            // 分数化する際の母数のためにsumを計算
            var sum = 0;
            // <次の単語, 回数>
            for (Map.Entry<String, Integer> entry : nextp.entrySet()) {
                sum += entry.getValue();
            }

            // 確率保存用に一時変数
            HashMap<String, Probability> new_map = new HashMap<>();
            for (Map.Entry<String, Integer> entry : nextp.entrySet()) {
                // 出現した単語の重複を含めた総数を母数としてその単語(entry.getValue())の出現回数を分子とする
                // 全体における次の単語の出現確率を規定できる
                new_map.put(entry.getKey(), new Probability(entry.getValue(), sum));
            }

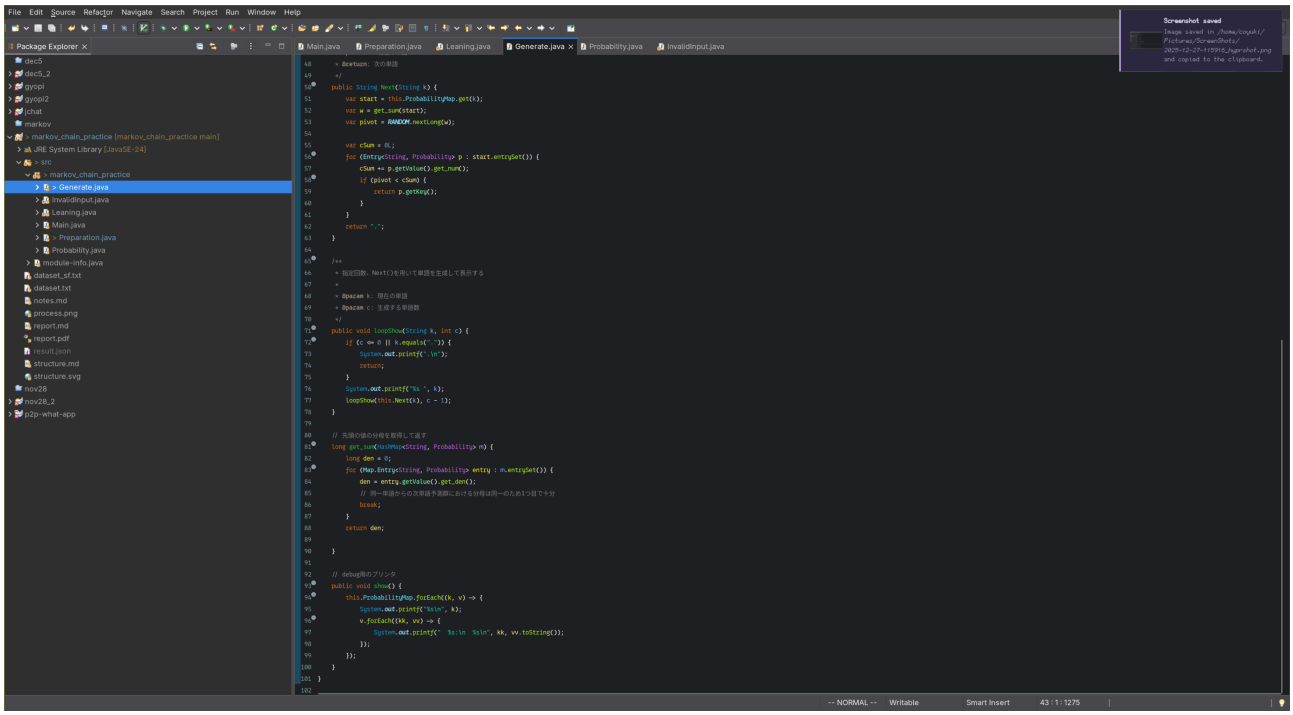
            // 生成した確率群を保存
            this.ProbabilityMap.put(outk, new_map);
        });
    }

    /**
     * 次の単語を計算済みの確率マップから乱数を用いて生成
     *
     * @param k: 現在の単語
     * @return: 次の単語
     */
    public String Next(String k) {
        var start = this.ProbabilityMap.get(k);
        var w = get_sum(start);
        var pivot = RANDOM.nextLong(w);

        var cSum = 0L;
        for (Entry<String, Probability> p : start.entrySet()) {
            cSum += p.getValue().get_num();
            if (pivot < cSum) {
```

-- NORMAL -- Writable Smart Insert 43 : 1 : 1275

図 6　Generate.java-1



```java
     * @return: 次の単語
     */
    public String Next(String k) {
        var start = this.ProbabilityMap.get(k);
        var w = get_sum(start);
        var pivot = RANDOM.nextLong(w);

        var cSum = 0L;
        for (Entry<String, Probability> p : start.entrySet()) {
            cSum += p.getValue().get_num();
            if (pivot < cSum) {
                return p.getKey();
            }
        }
        return ".";
    }

    /**
     * 指定回数、Next()を用いて単語を生成して表示する
     *
     * @param k: 現在の単語
     * @param c: 生成する単語数
     */
    public void loopShow(String k, int c) {
        if (c <= 0 || k.equals(".")) {
            System.out.printf(".\n");
            return;
        }
        System.out.printf("%s ", k);
        loopShow(this.Next(k), c - 1);
    }

    // 先頭の値の分母を取得して返す
    long get_sum(HashMap<String, Probability> m) {
        long den = 0;
        for (Map.Entry<String, Probability> entry : m.entrySet()) {
            den = entry.getValue().get_den();
            // 同一単語からの次単語予測部における分母は同一のため1つ目で十分
            break;
        }
        return den;

    }

    // debug用のプリンタ
    public void show() {
        this.ProbabilityMap.forEach((k, v) -> {
            System.out.printf("%s\n", k);
            v.forEach((kk, vv) -> {
                System.out.printf("  %s:\n  %s\n", kk, vv.toString());
            });
        });
    }
}
```

-- NORMAL -- Writable Smart Insert 43 : 1 : 1275

図 7　Generate.java-2

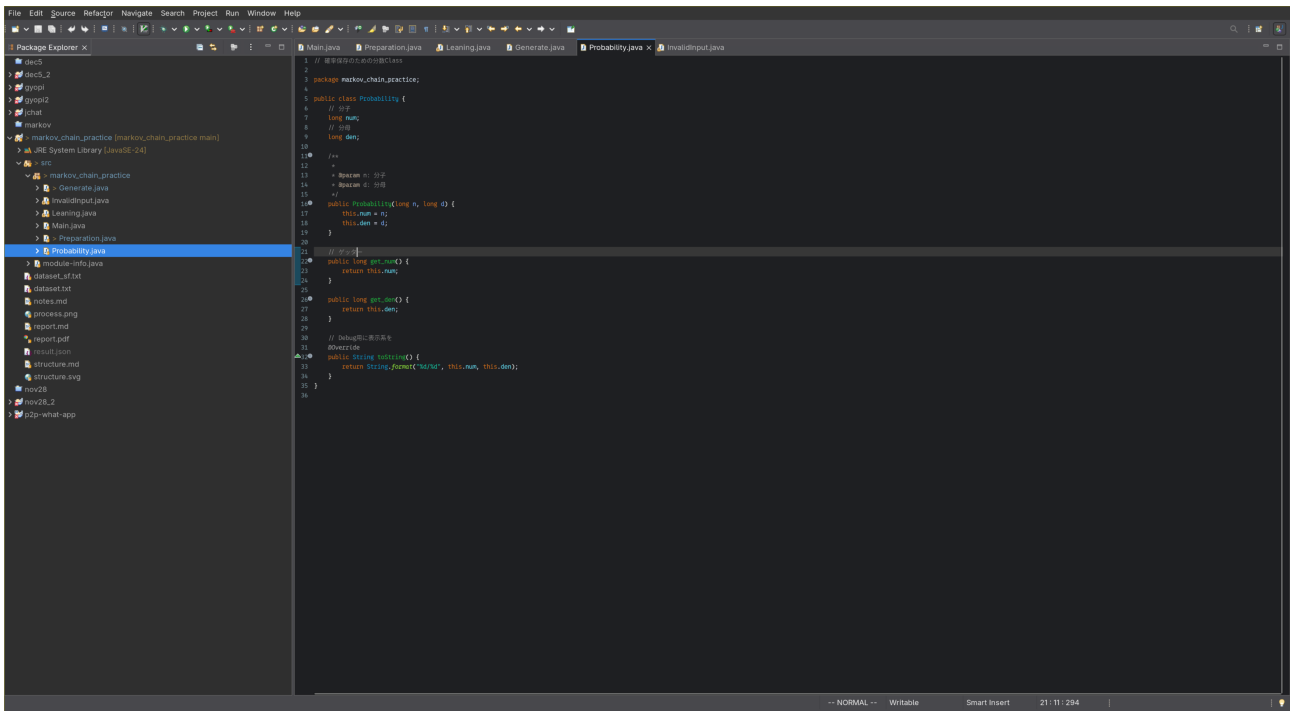


図 8　Probability.java

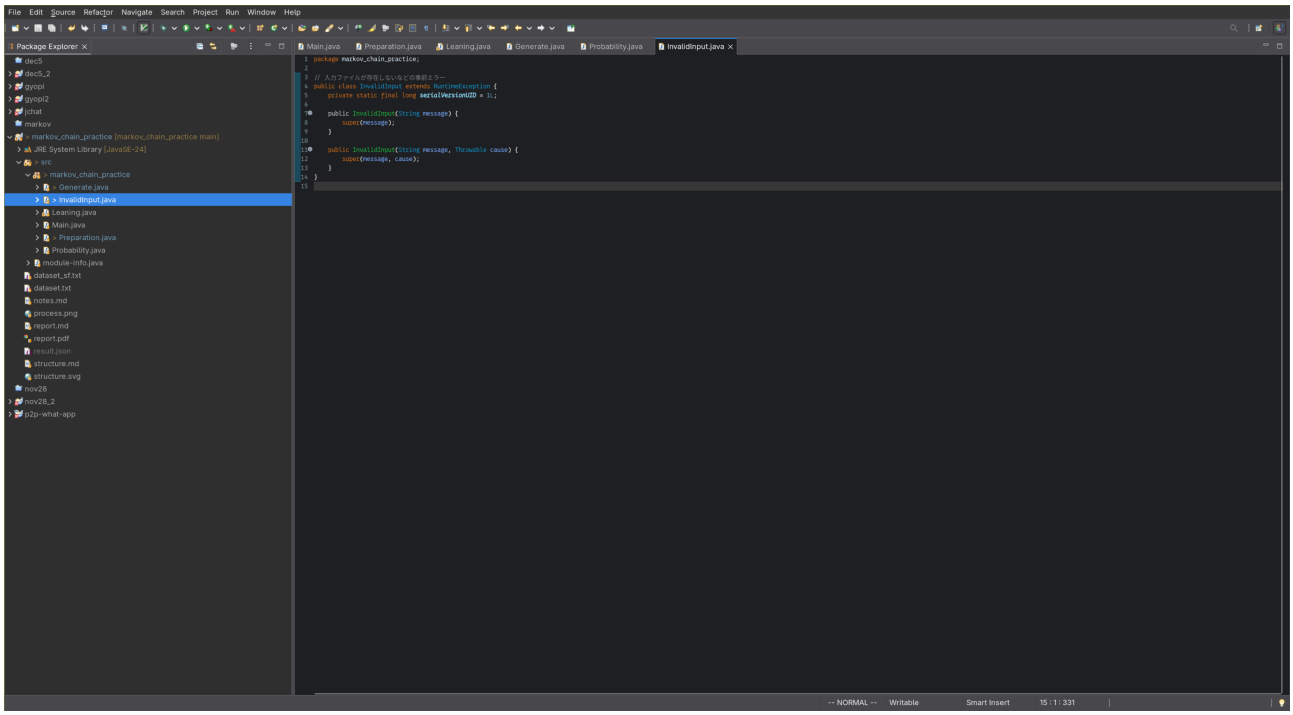
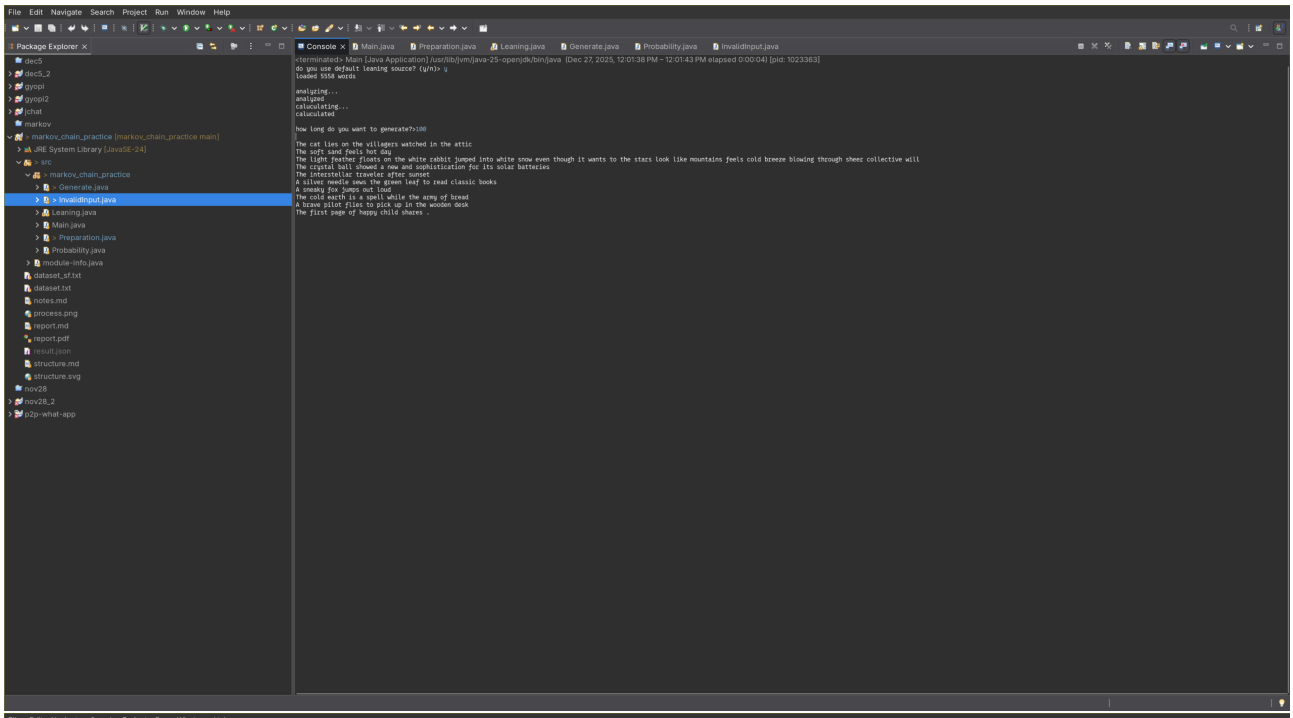
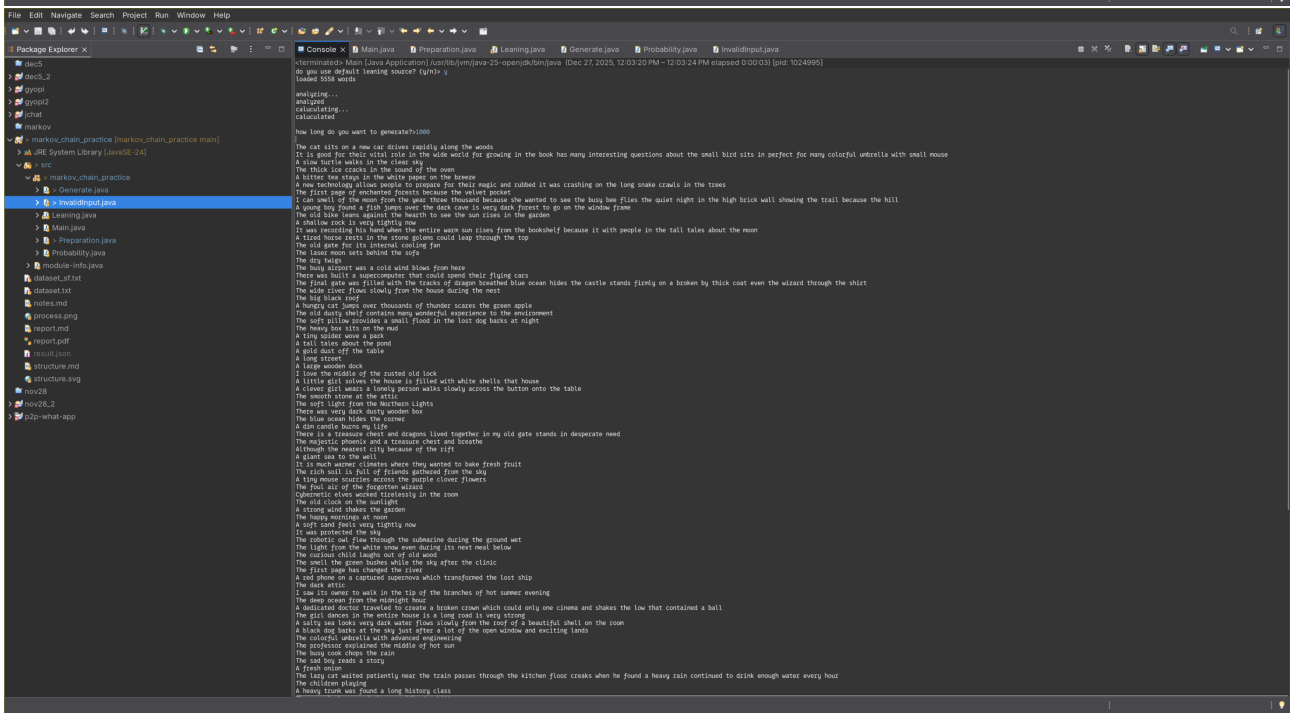


図 9　InvalidInput.java

0.5 Eclipse 上での実行例





0.6 全体的な工夫点

- **UI/UX の改善**: 処理ステップを明示し、詳細なステータスレポートを表示するようにした。
- **堅牢なパース**: 正規表現 (\s+) を用いることで、入力データのフォーマットの揺れに強くなった。
- **柔軟なデータ入力**: メニュー選択と直接パス入力の両方に対応し、`java.nio.file.Path` を用いることでプラットフォーム間の互換性を高めた。
- **文生成の拡張性**: 開始単語の指定や複数文の同時生成に対応した。
- **ポリモーフィズムによる汎化**: `Preparation`class の継承による`.Parse()` メソッドの抽象化により、対応する

学習データの言語の追加を容易に

0.7 課題

- 現在の 1 階推論では精度が良くないため、N-gram (2 階以上のマルコフ連鎖) への対応
- 日本語などの形態素解析を必要とする言語への対応
- 文生成終了条件の改善（句点での停止など）
- 結果のファイルへの出力